



cucumber-jvm@7.15.0 with Java HotSpot(TM) 64-Bit Server
VM@21.0.9+7-LTS-338 on Windows 11



100 % passed



3 minutes ago in 2 minutes 34 seconds



Search with text or @tags

✓ classpath:features/gorest/comments/CommentLifeCycle.feature

@Comments @FullCycle

Feature: Gestión del Ciclo de Vida de Comentarios

Como tester de la API de GoRest

Quiero ejecutar flujos de prueba de principio a fin (End-to-End)

Para asegurar la integridad referencial desde que un comentario nace hasta que se elimina.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario: CP01 Crear comentario con datos válidos para una publicación existente

- ✓ **Given** que preparo un comentario válido
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador del comentario
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear el comentario

Scenario: CP02 Consultar comentario existente de una publicación

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador del comentario
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear el comentario

Scenario: CP03 Modificar completamente un comentario con datos válidos

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **When** realizo una petición para modificar el comentario

- ✓ **Then** la API responde con un código de estado **200**
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al modificar el comentario

Scenario: CP04 Eliminar comentario existente de una publicación

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado **204**

Scenario: CP05 Verificar que el comentario eliminado ya no existe

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **And** realizo una petición para eliminar el comentario
- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado **404**
- ✓ **And** la respuesta contiene el mensaje **"Resource not found"**

Scenario: CP06 Intentar eliminar comentario eliminado previamente

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **And** realizo una petición para eliminar el comentario
- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado **404**
- ✓ **And** la respuesta contiene el mensaje **"Resource not found"**

✓ ✓ classpath:features/gorest/comments/CreateComment.feature

@Comments @CreateComment

Feature: Creación de Comentarios

Como tester de la API de GoRest

Quiero validar la creación de comentarios (POST /comments)

Para asegurar que solo se registran comentarios con datos válidos y tokens autorizados.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario Outline: CP01-CP02 Intentar crear un comentario con datos válidos para un usuario existente sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	mensaje_error
✓	ausente	Authentication failed
✓	inválido	Invalid token

Scenario Outline: CP03-CP06 Intentar crear un comentario con datos inválidos (Validación de campos) para una publicación existente

- ✓ **Given** que preparo un comentario con "<name>", "<email>" y "<body>"
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	name	email	body	campo	mensaje_error
✓		test@test.com	Contenido de prueba	name	can't be blank
✓	Name		Contenido de prueba	email	can't be blank
✓	Name	test@test.com		body	can't be blank
✓	Name	test_test.com	Contenido de prueba	email	is invalid

Scenario: CP05 Intentar crear un comentario para una publicación no existente

- ✓ **And** que preparo un comentario para una publicación con ID 999999
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "post" y el mensaje "must exist"

Scenario: CP06 Intentar crear un comentario con ID de publicación no existente y de tipo no numérico

- ✓ **When** realizo una petición para crear el comentario con el campo "post_id" informado con el valor "abc_123"
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "post" y el mensaje "must exist"
- ✓ **And** la respuesta de error contiene el campo "post_id" y el mensaje "is not a number"

Scenario: CP07 Intentar crear un comentario para una publicación eliminada previamente

- ✓ **And** realizo una petición para eliminar la publicación
- ✓ **And** la API responde con un código de estado 204
- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "post" y el mensaje "must exist"

Scenario: CP08 Crear comentario con datos válidos para una publicación existente

- ✓ **Given** que preparo un comentario válido
- ✓ **When** realizo una petición para crear el comentario
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador del comentario
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear el comentario

✓ classpath:features/gorest/comments/DeleteComment.feature

@Comments @DeleteComment

Feature: Eliminación de Comentarios

Como tester de la API de GoRest

Quiero validar la eliminación de comentarios (DELETE /comments/commentsId)

Para asegurar que los comentarios se eliminan correctamente y no queda rastro en el sistema.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario
- ✓ **And** que tengo un comentario creado para esa publicación

Scenario Outline: CP01-CP02 Intentar eliminar un comentario de una publicación sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP03 Intentar eliminar un comentario no existente

- ✓ **And** que utilizo un ID de "comentario" inexistente 999999
- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP04 Intentar eliminar un comentario eliminado previamente

- ✓ **And** realizo una petición para eliminar el comentario
- ✓ **And** la API responde con un código de estado 204
- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Eliminar un comentario existente

- ✓ **When** realizo una petición para eliminar el comentario
- ✓ **Then** la API responde con un código de estado 204

✓ classpath:features/gorest/comments/GetComment.feature

@Comments @GetComment

Feature: Consulta de Comentarios

Como tester de la API de GoRest

Quiero validar la consulta de comentarios (GET /comments/commentId)

Para asegurar que la información recuperada coincide con el recurso solicitado.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

- ✓ **And** que tengo un comentario creado para esa publicación

Scenario Outline: CP01-02 Intentar consultar un comentario sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP03 Intentar consultar un comentario no existente

- ✓ **Given** que utilizo un ID de "comentario" inexistente 999999
- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP04 Intentar consultar comentario eliminado previamente

- ✓ **Given** realizo una petición para eliminar el comentario
- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Consultar comentario existente de una publicación

- ✓ **When** realizo una petición para consultar el comentario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador del comentario

✓ ✓ classpath:features/gorest/comments/GetComments.feature

@Comments @GetComments

Feature: Consulta de comentarios

Como tester de la API de GoRest

Quiero validar la consulta de la lista de comentarios (GET /comments)

Para asegurar que el sistema responde correctamente a las peticiones del catálogo de comentarios.

Scenario Outline: CP01-CP02 Consultar lista de comentarios sin enviar token o enviando un token válido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición petición para consultar la lista de comentarios
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios no esta vacía
- ✓ **And** todos los comentarios tienen los campos id, post_id, name, email y body

Examples:

	tipo_token
✓	ausente
✓	válido

Scenario: CP03 Consultar lista de comentarios con token no válido

- ✓ **Given** que uso un tipo de autenticación "inválido"
- ✓ **When** realizo una petición petición para consultar la lista de comentarios
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "Invalid token"

✓ classpath:features/gorest/comments/ModifyComment.feature

@Comments @ModifyComment

Feature: Modificación de Comentarios

Como tester de la API de GoRest

Quiero validar la modificación de comentarios (PUT /comments/commentId, PATCH /comments/commentId)

Para asegurar que los cambios en el comentario se persisten correctamente.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario
- ✓ **And** que tengo un comentario creado para esa publicación

Scenario Outline: CP01-CP02 Intentar modificar completamente un comentario sin enviar token o enviando un token inválido

- ✓ **And** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para modificar el comentario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario Outline: CP03-CP04 Intentar modificar completamente un comentario con datos inválidos

- ✓ **Given** que preparo un comentario con "<name>", "<email>" y "<body>"
- ✓ **When** realizo una petición para modificar el comentario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	name	email	body	campo	mensaje_error
✓		test@test.com	Contenido de prueba	name	can't be blank
✓	Name		Contenido de prueba	email	can't be blank
✓	Name	test@test.com		body	can't be blank
✓	Name	test_test.com	Contenido de prueba	email	is invalid

Scenario: CP05 Intentar modificar completamente un comentario no existente

- ✓ **Given** que utilizo un ID de "comentario" inexistente 999999
- ✓ **When** realizo una petición para modificar el comentario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP06 Intentar modificar un comentario eliminado previamente

- ✓ **And** realizo una petición para eliminar el comentario
- ✓ **When** realizo una petición para modificar el comentario
- ✓ **Then** la API responde con un código de estado 404

- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP07 Modificar completamente un comentario con datos válidos

- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para modificar el comentario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador del comentario
- ✓ **And** la respuesta contiene los datos enviados en la petición al modificar el comentario

- ✓ classpath:features/gorest/nested/post_comments/CreatePostComment.feature

@Comments @CreatePostComment

Feature: Creación de Comentario en una Publicación

Como tester de la API de GoRest

Quiero validar la creación de comentarios en una publicación (POST /posts/postId/comments)

Para asegurar que solo se registran comentarios con datos válidos y tokens autorizados.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario Outline: CP01-CP02 Crear un comentario con datos válidos en una publicación existente sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para crear un comentario en la publicación
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	mensaje_error
✓	ausente	Authentication failed
✓	inválido	Invalid token

Scenario: CP03 Crear comentario para una publicación inexistente

- ✓ **Given** que utilizo un ID de "publicación" inexistente 999999
- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para crear un comentario en la publicación
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "post" y el mensaje "must exist"

Scenario: CP04 Crear comentario para una publicación eliminada previamente

- ✓ **Given** realizo una petición para eliminar la publicación
- ✓ **And** que preparo un comentario válido
- ✓ **When** realizo una petición para crear un comentario en la publicación
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "post" y el mensaje "must exist"

Scenario: CP05 Crear comentario para una publicación existente

- ✓ **Given** que preparo un comentario válido
- ✓ **When** realizo una petición para crear un comentario en la publicación
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador del comentario
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear el comentario

✓ classpath:features/gorest/nested/post_comments/GetPostComments.feature

@Comments @GetPostComments

Feature: Consulta de Comentarios de una Publicación

Como tester de la API de GoRest

Quiero validar la consulta de comentarios de una publicación (GET /posts/postId/comments)

Para asegurar que la información recuperada coincide con el recurso solicitado.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario: CP01 Consultar lista de comentarios de una publicación sin enviar token

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **And** que uso un tipo de autenticación "ausente"
- ✓ **When** realizo una petición para consultar los comentarios de la publicación

- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios esta vacía

Scenario: CP02 Consultar lista de comentarios con token no válido

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **And** que uso un tipo de autenticación "inválido"
- ✓ **When** realizo una petición para consultar los comentarios de la publicación
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "Invalid token"

Scenario: CP03 Consultar comentarios de una publicación recién creada

- ✓ **When** realizo una petición para consultar los comentarios de la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios esta vacía

Scenario: CP04 Consultar comentarios de una publicación inexistente

- ✓ **Given** que utilizo un ID de "publicación" inexistente 999999
- ✓ **When** realizo una petición para consultar los comentarios de la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios esta vacía

Scenario: CP05 Consultar comentarios de una publicación eliminada previamente

- ✓ **Given** realizo una petición para eliminar la publicación
- ✓ **When** realizo una petición para consultar los comentarios de la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios esta vacía

Scenario: CP06 Consultar comentarios de una publicación crear con un comentario

- ✓ **Given** que tengo un comentario creado para esa publicación
- ✓ **When** realizo una petición para consultar los comentarios de la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de comentarios no esta vacía
- ✓ **And** todos los comentarios tienen los campos id, post_id, name, email y body

✓ ✓ classpath:features/gorest/nested/user_posts/CreateUserPost.feature

@Posts @CreateUserPost

Feature: Creación de Publicaciones de un Usuario

Como tester de la API de GoRest

Quiero validar la creación de publicaciones de un usuario (POST /users/userId/posts)

Para asegurar que solo se registran publicaciones con datos válidos y tokens autorizados.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario Outline: CP01-CP02 Crear una publicación con datos válidos para un usuario existente sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para crear una publicación por el usuario
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	mensaje_error
✓	ausente	Authentication failed
✓	inválido	Invalid token

Scenario: CP03 Crear publicación para un usuario inexistente

- ✓ **Given** que utilizo un ID de "usuario" inexistente 999999
- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para crear una publicación por el usuario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "user" y el mensaje "must exist"

Scenario: CP04 Crear publicación para un usuario eliminado previamente

- ✓ **Given** realizo una petición para eliminar el usuario
- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para crear una publicación por el usuario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "user" y el mensaje "must exist"

Scenario: CP05 Crear publicación para un usuario

- ✓ **Given** que preparo una publicación válida
- ✓ **When** realizo una petición para crear una publicación por el usuario

- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear la publicación

✓ classpath:features/gorest/nested/user_posts/GetUserPosts.feature

@Posts @GetUserPosts

Feature: Consulta de Publicaciones de un Usuario

Como tester de la API de GoRest

Quiero validar la consulta de publicaciones de un usuario (GET /users/userId/posts)

Para asegurar que la información recuperada coincide con el recurso solicitado.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario: CP01 Consultar lista de publicaciones de un usuario sin enviar token

- ✓ **Given** que tengo una publicación creada para ese usuario
- ✓ **And** que uso un tipo de autenticación "ausente"
- ✓ **When** realizo una petición para consultar las publicaciones del usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones esta vacía

Scenario: CP02 Consultar lista de publicaciones de un usuario con token no válido

- ✓ **Given** que tengo una publicación creada para ese usuario
- ✓ **And** que uso un tipo de autenticación "inválido"
- ✓ **When** realizo una petición para consultar las publicaciones del usuario
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "Invalid token"

Scenario: CP03 Consultar publicaciones de un usuario recién creado

- ✓ **When** realizo una petición para consultar las publicaciones del usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones esta vacía

Scenario: CP04 Consultar publicaciones de un usuario inexistente

- ✓ **Given** que utilizo un ID de "usuario" inexistente 999999
- ✓ **When** realizo una petición para consultar las publicaciones del usuario

- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones esta vacía

Scenario: CP05 Consultar publicaciones de un usuario eliminado previamente

- ✓ **Given** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para consultar las publicaciones del usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones esta vacía

Scenario: CP06 Consultar publicaciones de un usuario creado con una publicación

- ✓ **Given** que tengo una publicación creada para ese usuario
- ✓ **When** realizo una petición para consultar las publicaciones del usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones no esta vacía

✓ ✓ classpath:features/gorest/posts/CreatePost.feature

@Posts @CreatePost

Feature: Creación de Publicaciones

Como tester de la API de GoRest

Quiero validar la creación de publicaciones (POST /posts)

Para asegurar que solo se registran publicaciones con datos válidos y tokens autorizados.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario Outline: CP01-CP02 Crear una publicación con datos válidos para un usuario existente sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	mensaje_error
✓	ausente	Authentication failed
✓	inválido	Invalid token

Scenario Outline: CP03-CP04 Intentar crear una publicación con datos inválidos (Validación de campos) para un usuario existente

- ✓ **Given** que preparo una publicación con "<title>" y "<body>"
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	title	body	campo	mensaje_error
✓		Contenido de prueba	title	can't be blank
✓	Title		body	can't be blank

Scenario: CP05 Intentar crear una publicación para un usuario que no existe

- ✓ **And** que preparo una publicación para un usuario con ID 999999
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "user" y el mensaje "must exist"

Scenario: CP06 Intentar crear una publicación con ID de usuario no numérico

- ✓ **When** envío una petición de creación con el campo "user_id" valor "abc_123"
- ✓ **Then** la API responde con un código de estado 422

Scenario: CP07 Intentar crear publicación para un usuario recién eliminado

- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **And** la API responde con un código de estado 204
- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 422

Scenario: CP05 Crear publicación con datos válidos para un usuario existente

- ✓ **Given** que preparo una publicación válida
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear la publicación

✓ classpath:features/gorest/posts/DeletePost.feature

@Posts @DeletePost

Feature: Eliminación de Publicaciones

Como tester de la API de GoRest

Quiero validar la eliminación de publicaciones (DELETE /posts/postId)

Para asegurar que las publicaciones se eliminan correctamente y no queda rastro en el sistema.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario Outline: CP01-CP02 Intentar eliminar una publicación de un usuario sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP03 Intentar eliminar una publicación no existente

- ✓ **Given** que utilizo un ID de "publicación" inexistente 999999
- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado 404

- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP04 Intentar eliminar publicación eliminada previamente

- ✓ **Given** realizo una petición para eliminar la publicación
- ✓ **And** la API responde con un código de estado 204
- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Eliminar una publicación existente

- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado 204

✓ ✓ classpath:features/gorest/posts/GetPost.feature

@Posts @GetPost

Feature: Consultar de Publicaciones

Como tester de la API de GoRest

Quiero validar la consulta de publicaciones (GET /posts/postId)

Para asegurar que la información recuperada coincide con el recurso solicitado.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario Outline: CP01-02 Intentar consultar una publicación sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para consultar la publicación
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP04 Intentar consultar una publicación no existente

- ✓ **Given** que utilizo un ID de "publicación" inexistente 999999
- ✓ **When** realizo una petición para consultar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Intentar consultar publicación eliminada previamente

- ✓ **Given** realizo una petición para eliminar la publicación
- ✓ **When** realizo una petición para consultar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP06 Consultar publicación existente de un usuario

- ✓ **When** realizo una petición para consultar la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador de la publicación

✓ classpath:features/gorest/posts/GetPosts.feature

@Posts @GetPosts

Feature: Consulta de publicaciones

Como tester de la API de GoRest

Quiero validar la consulta de la lista de publicaciones (GET /posts)

Para asegurar que el sistema responde correctamente a las peticiones del catálogo de publicaciones.

Scenario Outline: CP01-CP02 Consultar la lista de publicaciones sin enviar token o enviando un token válido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para consultar la lista de publicaciones
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de publicaciones no esta vacía
- ✓ **And** todas las publicaciones tienen los campos id, user_id, title y body

Examples:

	tipo_token
✓	ausente
✓	válido

Scenario: CP03 Intentar consultar la lista de publicaciones enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "inválido"
- ✓ **When** realizo una petición petición para consultar la lista de publicaciones
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "Invalid token"

✓ classpath:features/gorest/posts/ModifyPost.feature

@Posts @ModifyPost

Feature: Modificación de Publicaciones

Como tester de la API de GoRest

Quiero validar la modificación de publicaciones (PUT /posts/postId, PATCH /posts/postId)
Para asegurar que los cambios en la publicación se persisten correctamente.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que tengo una publicación creada para ese usuario

Scenario Outline: CP01-CP02 Intentar modificar completamente una publicación sin enviar token o enviando un token inválido

- ✓ **And** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario Outline: CP03-CP04 Intentar modificar completamente una publicación con datos inválidos

- ✓ **Given** que preparo una publicación con "<title>" y "<body>"
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado 422

- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	title	body	campo	mensaje_error
✓		Contenido de prueba	title	can't be blank
✓	Title		body	can't be blank

Scenario: CP05 Intentar modificar completamente una publicación no existente

- ✓ **Given** que utilizo un ID de "publicación" inexistente 999999
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP06 Intentar modificar una publicación eliminada previamente

- ✓ **And** realizo una petición para eliminar la publicación
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP07 Modificar completamente una publicación con datos válidos

- ✓ **And** que preparo una publicación válida
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al modificar la publicación

✓ classpath:features/gorest/posts/PostLifeCycle.feature

@Posts @FullCycle

Feature: Gestión del Ciclo de Vida de Publicaciones

Como tester de la API de GoRest

Quiero ejecutar flujos de prueba de principio a fin (End-to-End)

Para asegurar la integridad referencial desde que una publicación nace hasta que se elimina.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario: CP01 Crear publicación con datos válidos para un usuario existente

- ✓ **Given** que preparo una publicación válida
- ✓ **When** realizo una petición para crear la publicación
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al crear la publicación

Scenario: CP02 Consultar publicación existente de un usuario

- ✓ **Given** que preparo una publicación válida
- ✓ **And** realizo una petición para crear la publicación
- ✓ **When** realizo una petición para consultar la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador de la publicación

Scenario: CP03 Modificar completamente una publicación con datos válidos

- ✓ **Given** que preparo una publicación válida
- ✓ **And** realizo una petición para crear la publicación
- ✓ **When** realizo una petición para modificar la publicación
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene el identificador de la publicación
- ✓ **And** la respuesta contiene los datos enviados en la petición al modificar la publicación

Scenario: CP04 Eliminar publicación existente de un usuario

- ✓ **Given** que preparo una publicación válida
- ✓ **And** realizo una petición para crear la publicación
- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado 204

Scenario: CP05 Verificar que la publicación eliminada ya no existe

- ✓ **Given** que tengo una publicación creada para ese usuario
- ✓ **And** realizo una petición para eliminar la publicación
- ✓ **When** realizo una petición para consultar la publicación

- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP06 Intentar eliminar publicación eliminada previamente

- ✓ **Given** que tengo una publicación creada para ese usuario
- ✓ **And** realizo una petición para eliminar la publicación
- ✓ **When** realizo una petición para eliminar la publicación
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

✓ ✓ classpath:features/gorest/users/CreateUser.feature

@Users @CreateUser

Feature: Creación de Usuarios

Como tester de la API de GoRest

Quiero validar la creación de usuarios (POST /users/userId)

Para asegurar que solo se registran usuarios con datos válidos y tokens autorizados.

Scenario Outline: CP01-CP02 Intentar crear usuario con datos válidos sin enviar token o enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **And** que preparo un usuario válido
- ✓ **When** realizo una petición para crear un usuario
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	mensaje_error
✓	ausente	Authentication failed
✓	inválido	Invalid token

Scenario Outline: CP03-CP09 Intentar crear un usuario con datos inválidos

- ✓ **Given** que preparo un usuario con "<nombre>", "<email>", "<genero>" y "<estado>"
- ✓ **When** realizo una petición para crear un usuario
- ✓ **Then** la API responde con un código de estado 422

- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	nombre	email	genero	estado	campo	mensaje_error
✓		test@qaxpert.com	male	active	name	can't be blank
✓	Test		female	inactive	email	can't be blank
✓	Test	test@qaxpert.com		active	gender	can't be blank, can be male of female
✓	Test	test@qaxpert.com	male		status	can't be blank
✓	Test	email_sin_formato	female	active	email	is invalid
✓	Test	test@test.com	prefiere no decirlo	inactive	gender	can't be blank, can be male of female
✓	Test	test@test.com	male	ausente	status	can't be blank

Scenario: CP10 Intentar crear un usuario con un email ya registrado

- ✓ **Given** que preparo un usuario con el mismo email que el anterior
- ✓ **When** realizo una petición para crear un usuario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "email" y el mensaje "has already been taken"

Scenario: CP11 Crear usuario con datos válidos

- ✓ **And** que preparo un usuario válido
- ✓ **When** realizo una petición para crear un usuario
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene un identificador de usuario
- ✓ **And** la respuesta contiene los datos enviados en la petición

✓ classpath:features/gorest/users/DeleteUser.feature

@Users @DeleteUser

Feature: Eliminación de Usuarios

Como tester de la API de GoRest

Quiero validar la eliminación de usuarios (DELETE /users/userId)

Para asegurar que los usuarios se eliminan correctamente y no queda rastro en el sistema.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario Outline: CP01-CP02 Intentar eliminar un usuario sin enviar token o enviando un token inválido

- ✓ **And** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP03 Intentar eliminar un usuario no existente

- ✓ **And** que utilizo un ID de "usuario" inexistente 999999
- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP04 Intentar eliminar un usuario eliminado previamente

- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Eliminar usuario existente

- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado 204

✓ ✓ classpath:features/gorest/users/GetUser.feature

@Users @GetUser

Feature: Consulta de Usuarios

Como tester de la API de GoRest

Quiero validar la consulta de usuarios (GET /users/userId)

Para asegurar que la información recuperada coincide con el recurso solicitado.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario Outline: CP01-CP02 Intentar consultar un usuario sin enviar token o enviando un token inválido

- ✓ **And** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario: CP03 Intentar consultar un usuario no existente

- ✓ **And** que utilizo un ID de "usuario" inexistente 999999
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP04 Intentar consultar usuario eliminado previamente

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP05 Consultar usuario existente

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene un identificador de usuario

✓ classpath:features/gorest/users/GetUsers.feature

@Users @GetUsers

Feature: Consulta Lista de Usuarios

Como tester de la API de GoRest

Quiero validar la consulta de la lista de usuarios (GET /users)

Para asegurar que el sistema responde correctamente a las peticiones del catálogo de usuarios.

Scenario Outline: CP01-CP02 Consultar la lista de usuarios sin enviar token o enviando un token válido

- ✓ **Given** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición petición para consultar la lista de usuarios
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la lista de usuarios no esta vacía
- ✓ **And** todos los usuarios tienen los campos id, name, email, gender y status

Examples:

	tipo_token
✓	ausente
✓	válido

Scenario: CP03 Intentar consultar la lista de usuarios enviando un token inválido

- ✓ **Given** que uso un tipo de autenticación "inválido"
- ✓ **When** realizo una petición petición para consultar la lista de usuarios
- ✓ **Then** la API responde con un código de estado 401
- ✓ **And** la respuesta contiene el mensaje "Invalid token"

✓ classpath:features/gorest/users/ModifyUser.feature

@Users @ModifyUser

Feature: Modificación de Usuarios

Como tester de la API de GoRest

Quiero validar la modificación de usuarios (PUT /users/userId, PATCH /users/userId)

Para asegurar que los cambios en el perfil de usuario se persisten correctamente.

Background: Creación de precondiciones

- ✓ **Given** que tengo un usuario creado anteriormente

Scenario Outline: CP01-02 Intentar modificar completamente un usuario sin enviar token o enviando un token inválido

- ✓ **And** que uso un tipo de autenticación "<tipo_token>"
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado <codigo_estado>
- ✓ **And** la respuesta contiene el mensaje "<mensaje_error>"

Examples:

	tipo_token	codigo_estado	mensaje_error
✓	ausente	404	Resource not found
✓	inválido	401	Invalid token

Scenario Outline: CP03-CP09 Intentar modificar completamente un usuario con datos inválidos

- ✓ **Given** que preparo un usuario con "<nombre>", "<email>", "<genero>" y "<estado>"
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "<campo>" y el mensaje "<mensaje_error>"

Examples:

	nombre	email	genero	estado	campo	mensaje_error
✓		test@qaxpert.com	male	active	name	can't be blank
✓	Test		female	inactive	email	can't be blank
✓	Test	test@qaxpert.com		active	gender	can't be blank, can be male of female
✓	Test	test@qaxpert.com	male		status	can't be blank
✓	Test	email_sin_formato	female	active	email	is invalid
✓	Test	test@test.com	prefiere no decirlo	inactive	gender	can't be blank, can be male of female

	nombre	email	genero	estado	campo	mensaje_error
✓	Test	test@test.com	male	ausente	status	can't be blank

Scenario: CP10 Intentar modificar completamente un usuario no existente

- ✓ **Given** que utilizo un ID de "usuario" inexistente 999999
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP11 Intentar modificar un usuario completamente con un email ya registrado para otro usuario

- ✓ **And** que obtengo un email de otro usuario ya existente
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 422
- ✓ **And** la respuesta de error contiene el campo "email" y el mensaje "has already been taken"

Scenario: CP12 Intentar modificar un usuario eliminado previamente

- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP13 Modificar completamente un usuario con datos válidos

- ✓ **And** que preparo un usuario válido
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene un identificador de usuario
- ✓ **And** la respuesta contiene los datos enviados en la petición

✓ ✓ classpath:features/gorest/users/UserLifeCycle.feature

@Users @FullCycle

Feature: Gestión del Ciclo de Vida del Usuario

Como tester de la API de GoRest

Quiero ejecutar flujos de prueba de principio a fin (End-to-End)

Para asegurar la integridad referencial desde que un usuario nace hasta que se elimina.

Scenario: CP01 Crear usuario con datos válidos

- ✓ **And** que preparo un usuario válido
- ✓ **When** realizo una petición para crear un usuario
- ✓ **Then** la API responde con un código de estado 201
- ✓ **And** la respuesta contiene un identificador de usuario
- ✓ **And** la respuesta contiene los datos enviados en la petición

Scenario: CP02 Consultar usuario existente

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene un identificador de usuario

Scenario: CP03 Modificar completamente un usuario con datos válidos

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** que preparo un usuario válido
- ✓ **When** realizo una petición para modificar el usuario
- ✓ **Then** la API responde con un código de estado 200
- ✓ **And** la respuesta contiene un identificador de usuario
- ✓ **And** la respuesta contiene los datos enviados en la petición

Scenario: CP04 Eliminar usuario existente

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado 204

Scenario: CP05 Verificar que el usuario eliminado ya no existe

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para consultar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

Scenario: CP06 Intentar eliminar usuario eliminado previamente

- ✓ **Given** que tengo un usuario creado anteriormente
- ✓ **And** realizo una petición para eliminar el usuario
- ✓ **When** realizo una petición para eliminar el usuario
- ✓ **Then** la API responde con un código de estado 404
- ✓ **And** la respuesta contiene el mensaje "Resource not found"

